ThreatMatrix AI — Master Documentation v1.0

Part 4: ML Pipeline, LLM Integration & Data Strategy

> **Part:** 4 of 5 | **Version:** 1.0.0 | **Date:** 2026-02-23
> **Prev:** Part 3 — Modules | **Next:** Part 5 — Timeline

Table of Contents — Part 4

1. ML Strategy Overview
2. Benchmark Datasets
3. Feature Engineering Pipeline
4. Model 1: Isolation Forest
5. Model 2: Random Forest Classifier
6. Model 3: Autoencoder
7. Model Evaluation Framework
8. Real-Time Inference Pipeline
9. LLM Integration Architecture
10. LLM Provider Strategy & Budget
11. Threat Intelligence Data Strategy
12. Data Flow Architecture

---

1. ML Strategy Overview

1.1 The Three-Model Philosophy

The ML architecture deliberately employs **three fundamentally different** approaches to anomaly detection. This is not redundancy — it demonstrates deep CS understanding and serves different operational needs:

Model	Paradigm	Strength	Weakness	Operational Role
Isolation Forest	Unsupervised	Detects novel/unknown attacks (zero-day)	No attack classification	Anomaly sentinel — catches what others miss
Random Forest	Supervised	High accuracy on known attack types, explainable	Cannot detect unknown attacks	Primary classifier — categorizes known threats
Autoencoder	Deep Learning (Unsupervised)	Learns complex normal patterns, reconstruction-based	Slower inference, less explainable	Deep pattern detector — finds subtle deviations

1.2 Ensemble Scoring Strategy

The three models operate in parallel on every flow, producing a **composite anomaly score**:

```
                    ┌──────────────────┐
                    │   Network Flow   │
                    │  (Feature Vector) │
                    └──────────────────┘
                             │
                             │
              ┌──────────────┼──────────────┐
              ▼              ▼              ▼
       ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
       │ Isolation    │  │  Random      │  │ Auto-        │
       │ Forest       │  │  Forest      │  │ encoder      │
       │              │  │              │  │              │
       │ score: 0.87  │  │ label: DDoS  │  │ recon: 0.92  │
       │ anomaly: ☑   │  │ conf: 0.94   │  │ anomaly: ☑   │
       └──────┬───────┘  └──────┬───────┘  └──────┬───────┘
              │                 │                 │
              └─────────────────┼─────────────────┘
                                ▼
                  ┌──────────────────────┐
                  │  ENSEMBLE SCORER     │
                  │                      │
                  │  Composite Score:    │
                  │  w1×IF + w2×RF + w3×AE │
                  │  = 0.91              │
                  │                      │
                  │  Final Label: DDoS   │
                  │  Severity: CRITICAL  │
                  │  Confidence: 94%     │
                  └──────────────────────┘
                             │
                             ▼
                     ┌──────────────┐
                     │ Alert Engine │
                     └──────────────┘
```

Scoring Formula:

```
composite_score = (
    W_IF * isolation_forest_score +    # W_IF = 0.30 (anomaly detection weight)
    W_RF * random_forest_confidence +  # W_RF = 0.45 (classification weight)
    W_AE * autoencoder_recon_error     # W_AE = 0.25 (deep pattern weight)
)

# Thresholds
ALERT_CRITICAL = 0.90   # Composite ≥ 0.90 → Critical alert
ALERT_HIGH     = 0.75   # Composite ≥ 0.75 → High alert
ALERT_MEDIUM   = 0.50   # Composite ≥ 0.50 → Medium alert
ALERT_LOW      = 0.30   # Composite ≥ 0.30 → Low alert
```

---

2. Benchmark Datasets

2.1 Primary: NSL-KDD

The **gold standard** for academic IDS research. Required for academic credibility.

Attribute	Value
Full Name	NSL-KDD (Improved KDD Cup 99)
Records	125,973 (train) + 22,544 (test)
Features	41 features + 1 label
Attack Categories	4 classes: DoS, Probe, R2L, U2R
Specific Attacks	24 types: neptune, smurf, portsweep, satan, guess_passwd, etc.
Why Essential	Most cited IDS dataset in literature; reviewers expect it
Limitation	Dated (1999 traffic patterns), but academically mandatory
Source	University of New Brunswick (free download)

NSL-KDD Attack Distribution:

```
                    NSL-KDD LABEL DISTRIBUTION
Normal   ████████████████████████████████████  67,343 (53.5%)
DoS      ═══════════════════════               45,927 (36.5%)
Probe    ═══                                   11,656 (9.3%)
R2L      ═                                        995 (0.8%)
U2R      |                                         52 (0.04%)
```

2.2 Secondary: CICIDS2017

Modern, realistic dataset for contemporary attack validation.

Attribute	Value
Full Name	Canadian Institute for Cybersecurity IDS 2017
Records	~2.8 million flows
Features	80+ features (we select 40+ relevant)
Attack Types	Brute Force, DoS/DDoS, Web Attacks, Infiltration, Port Scan, Botnet
Duration	5 days of traffic (Monday-Friday)
Why Important	Modern attacks, realistic traffic mix, flow-based (matches our architecture)
Source	University of New Brunswick (free download)

2.3 Optional: UNSW-NB15

Attribute	Value

Attribute	Value
Records	~2.5 million
Attack Types	9 categories: Fuzzers, Analysis, Backdoor, DoS, Exploit, Generic, Recon, Shellcode, Worm
Use	Additional validation if time permits

2.4 Dataset Preparation Pipeline

```python
# Pseudocode for dataset preparation
def prepare_dataset(dataset_name: str) -> tuple[pd.DataFrame, pd.DataFrame]:
    """
    Standard pipeline applied to all datasets:
    1. Load raw data
    2. Handle missing values (drop or impute)
    3. Encode categorical features (Label Encoding + One-Hot)
    4. Normalize numerical features (StandardScaler)
    5. Feature selection (remove low-variance, correlations > 0.95)
    6. Stratified train/test split (80/20)
    7. Save processed data for reproducibility
    """
    raw = load_dataset(dataset_name)
    cleaned = handle_missing(raw)
    encoded = encode_categoricals(cleaned)
    normalized = scale_numericals(encoded)
    selected = select_features(normalized)
    X_train, X_test, y_train, y_test = train_test_split(
        selected, test_size=0.2, stratify=labels, random_state=42
    )
    return (X_train, y_train), (X_test, y_test)
```

---

3. Feature Engineering Pipeline

3.1 Feature Categories for Live Traffic

When capturing live VPS traffic, we extract features that **map to the benchmark dataset features** for model compatibility:

#	Feature	Type	Description	Dataset Mapping
1	`duration`	float	Flow duration in seconds	NSL-KDD: duration
2	`protocol_type`	cat	TCP/UDP/ICMP	NSL-KDD: protocol_type
3	`service`	cat	Destination service (http, dns, ssh, etc.)	NSL-KDD: service

#	Feature	Type	Description	Dataset Mapping
4	`flag`	cat	Connection status flag (SF, S0, REJ, etc.)	NSL-KDD: flag
5	`src_bytes`	int	Bytes from source to destination	NSL-KDD: src_bytes
6	`dst_bytes`	int	Bytes from destination to source	NSL-KDD: dst_bytes
7	`land`	bool	Source and destination IP/port are same	NSL-KDD: land
8	`wrong_fragment`	int	Number of wrong fragments	NSL-KDD: wrong_fragment
9	`urgent`	int	Number of urgent packets	NSL-KDD: urgent
10-15	`hot`, `num_failed_logins`, `logged_in`, `num_compromised`, `root_shell`, `su_attempted`	mixed	Content features (extracted from payload analysis)	NSL-KDD content features
16-22	`count`, `srv_count`, `serror_rate`, `srv_serror_rate`, `rerror_rate`, `srv_rerror_rate`, `same_srv_rate`	float	Time-based traffic features (2-second window)	NSL-KDD time features
23-30	`dst_host_count`, `dst_host_srv_count`, `dst_host_same_srv_rate`, etc.	float	Host-based traffic features (100-connection window)	NSL-KDD host features
31-40+	`entropy`, `mean_iat`, `std_iat`, `packets_per_sec`, `bytes_per_packet`, etc.	float	Extended features for CICIDS compatibility	CICIDS2017 features

3.2 Feature Extraction from Scapy

```python
# Core feature extraction from raw packets
class FlowFeatureExtractor:
    """
    Converts a collection of packets (one flow) into a feature vector.
    Maintains compatibility with NSL-KDD and CICIDS2017 feature spaces.
    """

    def extract(self, flow: Flow) -> dict:
        return {
            # Basic features
```

```
            'duration': flow.end_time - flow.start_time,
            'protocol_type': self._get_protocol(flow),
            'service': self._get_service(flow.dst_port),
            'flag': self._get_tcp_flag_status(flow),
            'src_bytes': flow.src_bytes,
            'dst_bytes': flow.dst_bytes,

            # Packet statistics
            'total_packets': flow.total_packets,
            'packets_per_second': flow.total_packets / max(flow.duration, 0.001),
            'bytes_per_packet': flow.total_bytes / max(flow.total_packets, 1),

            # Inter-arrival time statistics
            'mean_iat': np.mean(flow.inter_arrival_times),
            'std_iat': np.std(flow.inter_arrival_times),
            'min_iat': np.min(flow.inter_arrival_times) if
flow.inter_arrival_times else 0,
            'max_iat': np.max(flow.inter_arrival_times) if
flow.inter_arrival_times else 0,

            # TCP flag counts
            'syn_count': flow.syn_count,
            'ack_count': flow.ack_count,
            'fin_count': flow.fin_count,
            'rst_count': flow.rst_count,
            'psh_count': flow.psh_count,

            # Payload analysis
            'payload_entropy': self._calculate_entropy(flow.payload_bytes),
            'mean_payload_size': np.mean(flow.payload_sizes),

            # Connection behavior (windowed)
            'count': self._same_host_connections(flow, window=2),
            'srv_count': self._same_service_connections(flow, window=2),
            'serror_rate': self._syn_error_rate(flow),
            'rerror_rate': self._rej_error_rate(flow),

            # ... additional features
        }
```

---

4. Model 1: Isolation Forest

4.1 Algorithm Overview

Type: Unsupervised Anomaly Detection
Library: scikit-learn `IsolationForest`
Academic Value: Demonstrates understanding of ensemble methods, tree-based isolation, anomaly detection without labels

4.2 How It Works

```
Isolation Forest Principle:
━━━━━━━━━━━━━━━━━━━━━━━━━━━

Anomalies are FEW and DIFFERENT → they are EASIER TO ISOLATE

Normal points require MANY splits to isolate  → Long path length
Anomalies require FEW splits to isolate       → Short path length

Anomaly Score = f(average_path_length)
  → Score close to 1.0 = anomaly
  → Score close to 0.5 = normal
  → Score close to 0.0 = very normal (dense region)
```

4.3 Training Configuration

```python
from sklearn.ensemble import IsolationForest

model = IsolationForest(
    n_estimators=200,           # Number of isolation trees
    max_samples='auto',         # Subsample size
    contamination=0.05,         # Expected anomaly proportion (5%)
    max_features=1.0,           # Features per tree
    bootstrap=False,
    random_state=42,
    n_jobs=-1                   # Parallel training
)

# Train on NORMAL traffic only (unsupervised)
model.fit(X_train_normal)

# Score all test data
scores = model.decision_function(X_test)  # Lower = more anomalous
predictions = model.predict(X_test)       # 1 = normal, -1 = anomaly
```

4.4 Hyperparameter Tuning Strategy

Parameter	Range	Tuning Method
`n_estimators`	[100, 200, 300, 500]	Grid search on validation set
`contamination`	[0.01, 0.03, 0.05, 0.10]	Domain knowledge + cross-validation
`max_samples`	[256, 512, 1024, 'auto']	Subsampling efficiency
`max_features`	[0.5, 0.75, 1.0]	Feature diversity

4.5 Expected Performance (NSL-KDD)

Metric	Expected Range	Notes

Metric	Expected Range	Notes
Accuracy	88-93%	Lower than supervised due to no labels
Precision	85-91%	Some false positives on unusual normal traffic
Recall	90-95%	Good at catching anomalies
F1-Score	87-93%	Balanced performance
AUC-ROC	0.92-0.96	Strong discrimination
Inference	<1ms/sample	Very fast (tree traversal)

---

5. Model 2: Random Forest Classifier

5.1 Algorithm Overview

Type: Supervised Multi-Class Classification
Library: scikit-learn `RandomForestClassifier`
Academic Value: Demonstrates ensemble learning, feature importance, decision boundaries, multiclass classification

5.2 Training Configuration

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=300,           # Number of trees
    max_depth=30,               # Tree depth limit
    min_samples_split=5,        # Minimum samples to split
    min_samples_leaf=2,         # Minimum samples per leaf
    max_features='sqrt',        # Features per split
    class_weight='balanced',    # Handle class imbalance (R2L, U2R)
    criterion='gini',
    random_state=42,
    n_jobs=-1
)

# Train on labeled data (all classes)
# Classes: Normal, DoS, Probe, R2L, U2R (NSL-KDD)
# Classes: BENIGN, Bot, DDoS, DoS, FTP-Patator, ... (CICIDS2017)
model.fit(X_train, y_train)

# Predict with confidence
predictions = model.predict(X_test)
confidences = model.predict_proba(X_test)
```

5.3 Class Mapping (NSL-KDD → Alert Categories)

NSL-KDD Label	Attack Type	Alert Category	Examples
Normal	Benign traffic	—	Regular HTTP, DNS, SSH
DoS	Denial of Service	`ddos`	neptune, smurf, back, teardrop
Probe	Reconnaissance	`port_scan`	portsweep, ipsweep, nmap, satan
R2L	Remote to Local	`unauthorized_access`	guess_passwd, ftp_write, warezmaster
U2R	User to Root	`privilege_escalation`	buffer_overflow, rootkit, perl

5.4 Feature Importance Visualization

The top-10 most important features (Random Forest) — this is a key academic deliverable:

```
FEATURE IMPORTANCE (Random Forest on NSL-KDD)
═══════════════════════════════════════════

src_bytes          ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓  0.142
dst_bytes          ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓   0.138
service            ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓       0.112
flag               ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓        0.105
count              ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓          0.089
duration           ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓           0.076
dst_host_count     ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓            0.071
serror_rate        ▓▓▓▓▓▓▓▓▓▓▓▓▓▓             0.064
same_srv_rate      ▓▓▓▓▓▓▓▓▓▓▓▓▓              0.058
protocol_type      ▓▓▓▓▓▓▓▓▓▓▓▓               0.052
```

5.5 Expected Performance (NSL-KDD)

Metric	Expected Range	Notes
Accuracy	95-98%	Highest accuracy of the three models
Precision	94-97%	Excellent precision with balanced weights
Recall	96-99%	Very high recall for DoS/Probe
F1-Score	95-98%	Best overall F1
AUC-ROC	0.98-0.995	Near-perfect discrimination
Inference	1-2ms/sample	Fast (ensemble of decision trees)

---

6. Model 3: Autoencoder

6.1 Algorithm Overview

Type: Deep Learning — Unsupervised (Reconstruction-based Anomaly Detection)
Library: TensorFlow/Keras

Academic Value: Demonstrates neural networks, representation learning, non-linear dimensionality reduction, reconstruction error analysis

6.2 Architecture

```
INPUT (41 features)
    │
    ▼
┌──────────────────┐
│ Dense(64, relu)  │  ── Encoder
│ BatchNorm        │
│ Dropout(0.2)     │
├──────────────────┤
│ Dense(32, relu)  │  ── Encoder
│ BatchNorm        │
│ Dropout(0.2)     │
├──────────────────┤
│ Dense(16, relu)  │  ── Bottleneck (Latent Space)
├──────────────────┤
│ Dense(32, relu)  │  ── Decoder
│ BatchNorm        │
│ Dropout(0.2)     │
├──────────────────┤
│ Dense(64, relu)  │  ── Decoder
│ BatchNorm        │
├──────────────────┤
│ Dense(41, sigmoid)│  ── Output (Reconstruction)
└──────────────────┘

Loss: MSE between input and reconstruction
Anomaly: High reconstruction error → Anomaly
```

6.3 Training Configuration

```python
import tensorflow as tf
from tensorflow import keras

# Autoencoder architecture
encoder_input = keras.Input(shape=(n_features,))
x = keras.layers.Dense(64, activation='relu')(encoder_input)
x = keras.layers.BatchNormalization()(x)
x = keras.layers.Dropout(0.2)(x)
x = keras.layers.Dense(32, activation='relu')(x)
x = keras.layers.BatchNormalization()(x)
x = keras.layers.Dropout(0.2)(x)
bottleneck = keras.layers.Dense(16, activation='relu')(x)

x = keras.layers.Dense(32, activation='relu')(bottleneck)
x = keras.layers.BatchNormalization()(x)
x = keras.layers.Dropout(0.2)(x)
```

```
x = keras.layers.Dense(64, activation='relu')(x)
x = keras.layers.BatchNormalization()(x)
decoder_output = keras.layers.Dense(n_features, activation='sigmoid')(x)

autoencoder = keras.Model(encoder_input, decoder_output)
autoencoder.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='mse'
)

# Train on NORMAL traffic only
autoencoder.fit(
    X_train_normal, X_train_normal,   # Input = Target (reconstruction)
    epochs=100,
    batch_size=256,
    validation_split=0.15,
    callbacks=[
        keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True),
        keras.callbacks.ReduceLROnPlateau(factor=0.5, patience=5)
    ]
)

# Anomaly detection via reconstruction error
reconstructions = autoencoder.predict(X_test)
mse = np.mean(np.power(X_test - reconstructions, 2), axis=1)
threshold = np.percentile(train_mse, 95)  # 95th percentile of training error
predictions = (mse > threshold).astype(int)  # 1 = anomaly
```

6.4 Reconstruction Error Distribution

```
RECONSTRUCTION ERROR DISTRIBUTION
═══════════════════════════════════

           Normal Traffic          Attack Traffic
           ┌──────────────┐        ┌──────────────────────┐
Count ▓▓▓▓▓│▓▓▓▓▓▓▓▓▓▓▓▓▓ │        │     ▓▓▓▓             │
      ▓▓▓▓▓│▓▓▓▓▓▓▓▓▓▓▓▓▓ │        │   ▓▓▓▓▓▓▓▓▓▓         │
      ▓▓▓▓▓│▓▓▓▓▓▓▓▓▓▓▓▓▓ │        │ ▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓     │
      ▓▓▓▓▓│▓▓▓▓▓▓▓▓▓▓▓▓▓ │        │▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓▓│
      ─────┴──────────────┴────────┴──────────────────────┴────
           0.001   0.01   ↑0.05    0.1        0.5       1.0
                          │
                       THRESHOLD
                  (95th percentile)
```

6.5 Expected Performance (NSL-KDD)

Metric	Expected Range	Notes
Accuracy	93-96%	Strong with proper threshold tuning

Metric	Expected Range	Notes
Precision	91-95%	Good but threshold-dependent
Recall	93-97%	Catches complex patterns
F1-Score	92-96%	Strong overall
AUC-ROC	0.95-0.98	Excellent discrimination
Inference	2-5ms/sample	Slower (neural network forward pass)
Training	30-120 seconds	GPU recommended but CPU acceptable

---

7. Model Evaluation Framework

7.1 Evaluation Metrics Computed for Each Model

Metric	Formula	Purpose
Accuracy	(TP+TN)/(TP+TN+FP+FN)	Overall correctness
Precision	TP/(TP+FP)	How many predicted positives are correct
Recall (Sensitivity)	TP/(TP+FN)	How many actual positives are caught
F1-Score	2×(P×R)/(P+R)	Harmonic mean of precision and recall
AUC-ROC	Area under ROC curve	Discrimination ability across thresholds
Confusion Matrix	NxN matrix	Per-class classification details
Training Time	Seconds	Computational efficiency
Inference Time	Milliseconds/sample	Real-time feasibility
Feature Importance	Model-specific	Interpretability (RF only)

7.2 Cross-Validation Strategy

```
from sklearn.model_selection import StratifiedKFold

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# For each fold:
#   1. Train model on 4 folds
#   2. Evaluate on held-out fold
#   3. Record all metrics
#   4. Report mean ± std across 5 folds
```

7.3 The Academic Deliverable: Model Comparison Table

This table is the **centerpiece of the academic submission**:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MODEL PERFORMANCE COMPARISON                          │
│                    Dataset: NSL-KDD Test Set (22,544 samples)            │
├──────────────┬──────────────┬──────────────┬──────────────┬──────────────┤
│ Metric       │ Isolation    │ Random       │ Autoencoder  │ Ensemble     │
│              │ Forest       │ Forest       │              │ (Weighted)   │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Accuracy     │ 91.2 ± 0.8%  │ 96.8 ± 0.3%  │ 94.5 ± 0.5%  │ 97.3 ± 0.2%  │
│ Precision    │ 88.5 ± 1.1%  │ 96.1 ± 0.4%  │ 93.0 ± 0.7%  │ 96.8 ± 0.3%  │
│ Recall       │ 93.2 ± 0.6%  │ 97.5 ± 0.2%  │ 95.8 ± 0.4%  │ 97.9 ± 0.2%  │
│ F1-Score     │ 90.8 ± 0.9%  │ 96.8 ± 0.3%  │ 94.4 ± 0.5%  │ 97.3 ± 0.2%  │
│ AUC-ROC      │ 0.943 ± .008 │ 0.990 ± .002 │ 0.967 ± .005 │ 0.993 ± .001│
│ Train Time   │ 2.3s         │ 8.7s         │ 45.2s        │ 56.2s        │
│ Infer (ms)   │ 0.8          │ 1.2          │ 3.5          │ 5.5          │
│ Zero-Day     │ ☑            │ ✗            │ ☑            │ ☑            │
│ Explainable  │ Partial      │ ☑ Full       │ ✗ Limited    │ Partial      │
├──────────────┴──────────────┴──────────────┴──────────────┴──────────────┤
│ Champion: Random Forest (highest F1)                                     │
│ Sentinel: Isolation Forest (zero-day detection)                          │
│ Ensemble: Best overall metrics (weighted combination)                    │
└──────────────────────────────────────────────────────────────────────────┘
```

---

8. Real-Time Inference Pipeline

8.1 Pipeline Architecture

```
VPS Network Interface
        │
        ▼
┌─────────────────┐     ┌─────────────────┐     ┌─────────────────┐
│ Scapy Capture   │────►│ Flow Aggregator │────►│ Feature         │
│ (raw packets)   │     │ (5-tuple buffer)│     │ Extractor       │
└─────────────────┘     └─────────────────┘     └─────────┬───────┘
                                                          │
                                                          ▼
                                                  ┌─────────────────┐
                                                  │ Feature Vector  │
                                                  │ (40+ values)    │
                                                  └─────────┬───────┘
                                                            │
                                          ┌─────────────────┴─────────────────┐
                                          │                                   │
                                          ▼                                   ▼
                                  ┌─────────────────┐                 ┌─────────────────┐
                                  │ Redis Pub/Sub   │                 │ PostgreSQL      │
                                  │ (flows:live)    │                 │ (persist flow)  │
                                  └─────────┬───────┘                 └─────────────────┘
                                            │
                                            ▼
```

```
                ┌──────────────────┐
                │ ML Worker        │
                │ (subscribes to   │
                │  flows:live)     │
                │                  │
                │ 1. Deserialize   │
                │ 2. Normalize     │
                │ 3. Run 3 models  │
                │ 4. Ensemble      │
                │ 5. Score flow    │
                └──────────────────┘
                         │
                         │
            ┌────────────┼────────────┐
            ▼            ▼            ▼

      ┌────────────┐ ┌─────────┐ ┌──────────────┐
      │ Update     │ │ Create  │ │ Redis Pub/Sub│
      │ flow in    │ │ Alert   │ │ (alerts:live)│
      │ PostgreSQL│ │ if      │ │ → WebSocket  │
      │ (scores)   │ │ needed  │ │ → Browser    │
      └────────────┘ └─────────┘ └──────────────┘
```

8.2 Performance Targets

Stage	Latency Target	Strategy
Packet capture → flow complete	30-120 seconds	Flow timeout configuration
Feature extraction	<10ms	Optimized numpy operations
ML inference (3 models)	<10ms	Pre-loaded models in memory
Ensemble scoring	<1ms	Simple weighted average
Alert creation + broadcast	<50ms	Redis pub/sub + async DB write
Total: Flow complete → Alert visible	**<200ms**	**Well within real-time**

8.3 Model Loading Strategy

```python
# Models loaded once at worker startup, kept in memory
class ModelManager:
    def __init__(self):
        self.isolation_forest = joblib.load('models/isolation_forest_v1.pkl')
        self.random_forest = joblib.load('models/random_forest_v1.pkl')
        self.autoencoder = keras.models.load_model('models/autoencoder_v1.h5')
        self.scaler = joblib.load('models/standard_scaler.pkl')

    def predict(self, features: np.ndarray) -> dict:
        scaled = self.scaler.transform(features.reshape(1, -1))

        if_score = -self.isolation_forest.decision_function(scaled)[0]
        rf_proba = self.random_forest.predict_proba(scaled)[0]
```

```
        rf_label = self.random_forest.classes_[np.argmax(rf_proba)]
        rf_conf = np.max(rf_proba)
        ae_recon = self.autoencoder.predict(scaled, verbose=0)
        ae_error = np.mean(np.power(scaled - ae_recon, 2))

        composite = (0.30 * if_score + 0.45 * rf_conf + 0.25 * min(ae_error * 10,
1.0))

        return {
            'composite_score': composite,
            'isolation_forest': {'score': if_score, 'is_anomaly': if_score > 0.5},
            'random_forest': {'label': rf_label, 'confidence': rf_conf,
'probabilities': rf_proba.tolist()},
            'autoencoder': {'reconstruction_error': ae_error, 'is_anomaly':
ae_error > self.ae_threshold},
            'is_anomaly': composite > 0.30,
            'severity': self._score_to_severity(composite),
        }
```

---

9. LLM Integration Architecture

9.1 LLM Gateway Service

The LLM Gateway is a **centralized service** that manages all LLM interactions with provider routing, caching, budget tracking, and fallback logic.

```
┌──────────────────────────────────────────────────────────────────┐
│                    LLM GATEWAY SERVICE                           │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │              REQUEST ROUTER                              │   │
│  │                                                          │   │
│  │  Task Type → Provider Selection:                         │   │
│  │  • Complex Analysis    → DeepSeek V3 (best reasoning)    │   │
│  │  • Real-time Alerts    → Groq Llama 3.3 (fastest)        │   │
│  │  • Bulk/Translation    → GLM-4-Flash (cheapest)          │   │
│  │  • Fallback            → Next available provider         │   │
│  └──────────┬──────────────┬──────────────┬──────────────────┘   │
│             │              │              │                  │
│        ┌────▼────┐    ┌────▼────┐    ┌────▼────┐            │
│        │ DeepSeek │    │ Groq    │    │ GLM (Zhipu) │            │
│        │ V3       │    │ Llama   │    │ 4-Flash     │            │
│        │          │    │ 3.3 70B │    │             │            │
│        │ $0.14/M  │    │ $0.06/M │    │ $0.01/M     │            │
│        │ in tokens │    │ tokens  │    │ tokens      │            │
│        └──────────┘    └─────────┘    └─────────────┘            │
│                                                                  │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │  MIDDLEWARE STACK                                        │   │
│  │  1. Budget Check    → Reject if monthly budget exceeded  │   │
```

```
│  │  2. Cache Lookup    → Return cached if identical query  │ │
│  │  3. Rate Limiter    → Max 20 req/min per user           │ │
│  │  4. Prompt Builder  → Construct system + user prompt    │ │
│  │  5. Token Counter   → Track input/output tokens         │ │
│  │  6. Response Cache  → Cache response (TTL: 1 hour)      │ │
│  │  7. Cost Logger     → Log cost to PostgreSQL            │ │
│  └──────────────────────────────────────────────────────────┘ │
└────────────────────────────────────────────────────────────────┘
```

9.2 Prompt Templates

```
PROMPTS = {
    "alert_analysis": """
You are ThreatMatrix AI Analyst, an expert cybersecurity analyst.
Analyze the following network security alert and provide:
1. A clear explanation of what happened
2. Why this is dangerous
3. Recommended immediate actions
4. Long-term remediation steps

Alert Details:
- ID: {alert_id}
- Severity: {severity}
- Category: {category}
- Source IP: {source_ip} → Destination IP: {dest_ip}
- ML Confidence: {confidence}%
- Related flows: {flow_count}
- Timestamp: {timestamp}

Flow Statistics:
{flow_summary}

Threat Intel Matches:
{intel_matches}

Provide your analysis in a clear, professional format.
""",

    "daily_briefing": """
You are ThreatMatrix AI, generating a daily cyber threat briefing.

Network Statistics (Last 24 Hours):
- Total flows analyzed: {total_flows}
- Anomalous flows detected: {anomaly_count} ({anomaly_pct}%)
- Alerts generated: {alert_count}
  - Critical: {critical}, High: {high}, Medium: {medium}, Low: {low}
- Top attack categories: {top_categories}
- Top source IPs (suspicious): {top_sources}
- Top targeted ports: {top_ports}

Current Threat Level: {threat_level}
```

```
Generate a concise executive briefing covering:
1. Overall threat posture assessment
2. Key incidents and patterns
3. Recommendations for the security team
4. Predicted risk trend for next 24 hours
""",

    "ip_investigation": """
Investigate the following IP address for potential threats:

IP: {ip_address}
Internal observations:
- Flows involving this IP: {flow_count}
- Anomalous flows: {anomaly_count}
- Protocols used: {protocols}
- Ports accessed: {ports}
- First seen: {first_seen}
- Last seen: {last_seen}

External intelligence:
{intel_data}

Provide a risk assessment with confidence level.
""",

    "amharic_translation": """
Translate the following cybersecurity alert/report to Amharic (አማርኛ).
Maintain technical terms in English where Amharic equivalents don't exist.
Keep the professional tone.

Text to translate:
{text}
"""
}
```

9.3 Streaming Response Implementation

```python
# FastAPI streaming endpoint for chat
from fastapi.responses import StreamingResponse

@router.post("/llm/chat")
async def chat(request: ChatRequest, user: User = Depends(get_current_user)):
    async def stream_response():
        async for token in llm_gateway.stream_chat(
            messages=request.messages,
            task_type="general_analysis",
            user_id=user.id
        ):
            yield f"data: {json.dumps({'token': token})}\n\n"
        yield "data: [DONE]\n\n"
```

```
    return StreamingResponse(stream_response(), media_type="text/event-stream")
```

---

10. LLM Provider Strategy & Budget

10.1 Provider Comparison

Provider	Model	Speed	Quality	Cost/M tokens	Best For
DeepSeek	V3	Medium	★★★★★	$0.14 in / $0.28 out	Complex analysis, reasoning
Groq	Llama 3.3 70B	★★★★★ (fastest)	★★★★	$0.06	Real-time alerts, quick queries
GLM (Zhipu)	GLM-4-Flash	Fast	★★★★	$0.01	Bulk tasks, translations, classification
Together AI	Llama 3.1 8B	Fast	★★★	$0.02	Fallback, simple tasks

10.2 Budget Allocation (Total: $100-200)

Category	Allocation	Provider	Est. Requests
Complex Analysis	$40-60	DeepSeek V3	~3,000-4,000 analyses
Real-time Alerts	$20-30	Groq	~5,000-8,000 summaries
Bulk/Translation	$15-25	GLM-4-Flash	~50,000+ operations
Demo Day Reserve	$30-50	All providers	Buffer for live demo
Total	**$105-165**		

10.3 Cost Per Feature

Feature	Est. Tokens/Request	Provider	Cost/Request	Daily Usage	Daily Cost
Alert Narrative	~500 in + 300 out	DeepSeek	$0.00015	50 alerts	$0.0075
Chat Query	~800 in + 500 out	DeepSeek	$0.00025	20 queries	$0.005
Daily Briefing	~1500 in + 800 out	DeepSeek	$0.00044	2 briefings	$0.0009
Quick Summary	~300 in + 200 out	Groq	$0.00003	100/day	$0.003
Amharic Translation	~500 in + 500 out	GLM	$0.00001	30/day	$0.0003
Total Daily					**~$0.017**
Total Monthly					**~$0.50**

At $0.50/month, the $100-200 budget covers **the entire development + demo period with massive headroom**.

---

11. Threat Intelligence Data Strategy

11.1 Feed Integration Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                  THREAT INTEL AGGREGATOR                          │
│                                                                   │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐   │
│  │ AlienVault   │  │ AbuseIPDB    │  │ VirusTotal           │   │
│  │ OTX          │  │              │  │                      │   │
│  │ Free: ∞      │  │ Free: 1K/day │  │ Free: 500/day        │   │
│  │              │  │              │  │                      │   │
│  │ Pulses,      │  │ IP reports,  │  │ IP/domain/hash       │   │
│  │ IOCs,        │  │ confidence,  │  │ analysis,            │   │
│  │ threat       │  │ categories   │  │ detection ratios     │   │
│  │ descriptions│  │              │  │                      │   │
│  └──────────────┘  └──────────────┘  └──────────────────────┘   │
│          │                │                      │               │
│          └────────────────┼──────────────────────┘               │
│                           ▼                                      │
│                  ┌──────────────┐                                │
│                  │ Normalizer   │                                │
│                  │ • Deduplicate │                               │
│                  │ • Score merge │                               │
│                  │ • Tag enrich  │                               │
│                  └──────────────┘                                │
│                           ▼                                      │
│                  ┌──────────────┐                                │
│                  │ PostgreSQL   │                                │
│                  │ threat_intel │                                │
│                  │ _iocs table  │                                │
│                  └──────────────┘                                │
│                           ▼                                      │
│                  ┌──────────────┐                                │
│                  │ Correlator   │  ← Matches IOCs against        │
│                  │              │     live network flows          │
│                  └──────────────┘                                │
└──────────────────────────────────────────────────────────────────┘
```

11.2 Sync Schedule

Feed	Sync Frequency	Method	Data Volume per Sync
OTX	Every 6 hours	Pull latest pulses via API	~500-2000 IOCs
AbuseIPDB	On-demand (when IP seen in traffic)	Real-time lookup	1 IP per query

Feed	Sync Frequency	Method	Data Volume per Sync
VirusTotal	On-demand (when IOC flagged)	Real-time lookup	1 item per query

11.3 Correlation Engine

When a new flow is analyzed, the Correlator checks:

1. Is `src_ip` or `dst_ip` in the IOC database? → If yes, auto-escalate alert severity
2. Is the `dst_domain` (from DNS) in the IOC database? → If yes, flag as C2/phishing
3. Are any file hashes (if payload extracted) in VirusTotal? → If yes, flag as malware

---

12. Data Flow Architecture

12.1 Complete Data Flow (End-to-End)

```
NETWORK INTERFACE
       │
       ▼
[1] CAPTURE ENGINE (Scapy)
    • Sniffs packets on VPS network interface
    • Groups into flows by 5-tuple
    • Extracts 40+ features per flow
       │
       ├──► [2] Redis Pub/Sub (channel: flows:live)
       │         └──► ML Worker subscribes
       │
       └──► [3] PostgreSQL (network_flows table)
                • Persists all flow records
       │
[4] ML WORKER (Background Process)
    • Receives flow from Redis
    • Runs Isolation Forest → score
    • Runs Random Forest → label + confidence
    • Runs Autoencoder → reconstruction error
    • Computes ensemble composite score
       │
       ├──► [5] PostgreSQL UPDATE network_flows SET anomaly_score, is_anomaly,
label
       │
       └──► [6] IF composite_score > threshold:
                • CREATE alert in PostgreSQL
                • PUBLISH to Redis (channel: alerts:live)
                • REQUEST LLM narrative (async, via task queue)
                    │
                    └──► [7] LLM Gateway
                         • Select provider (DeepSeek/Groq/GLM)
                         • Generate threat narrative
                         • UPDATE alert.ai_narrative
                         │
[8] FASTAPI WEBSOCKET SERVER
```

```
      • Subscribes to Redis channels (flows:live, alerts:live)
      • Broadcasts to connected browser clients
        │
        ▼
  [9] NEXT.JS COMMAND CENTER (Browser)
      • Receives WebSocket events
      • Updates War Room in real-time
      • Renders alerts, maps, charts
      • AI Analyst chat interface
```

12.2 Data Retention Policy

Data Type	Retention	Storage Est. (30 days)	Cleanup
Network Flows	90 days	~2-5 GB	Cron job: DELETE WHERE created_at < NOW() - 90d
Alerts	1 year	~100 MB	Archive to cold storage after 90 days
Threat Intel IOCs	Indefinite	~500 MB	Mark stale after 180 days
LLM Conversations	90 days	~200 MB	Auto-cleanup
PCAP Uploads	30 days	~1-10 GB (varies)	Delete processed files after 30 days
ML Models	Indefinite	~500 MB	Keep all versions for comparison
Audit Logs	1 year	~100 MB	Archive after 6 months

> **End of Part 4** — Continue to Part 5: Development Timeline, Team Workflow & Deployment Guide

---

ThreatMatrix AI Master Documentation v1.0 — Part 4 of 5
© 2026 ThreatMatrix AI. All rights reserved.